Sant'Anna
Scuola Universitaria Superiore Pisa

Sant'Anna School of Advanced Studies, Pisa

Course: **PDAI2: Programming, Data Analytics and Artificial Intelligence**

March 2nd, 2026

An Introduction to **Large Language Models**

Course Responsible: **prof. Andrea Vandin**

Lecturer: **Lorenzo Emer**

Today’s Lecture

- **Background**: From Encoder–Decoder Models to Modern LLMs
- **How LLMs Work**: Attention, and Next-Token Prediction
- **How to Make LLMs Work**: Prompt Engineering
- **Grounding and Adaptation**: Retrieval-Augmented Generation and Fine-Tuning
- **LLMs in Practice:** Hands-on session in Python

What is a **Large Language Model**?

A Large Language Model (LLM) is a **neural network (NN)** trained on **massive text corpora** to estimate the **probability of the next token** given previous tokens. In other words, it is a NN that generates text conditioned on previous text.

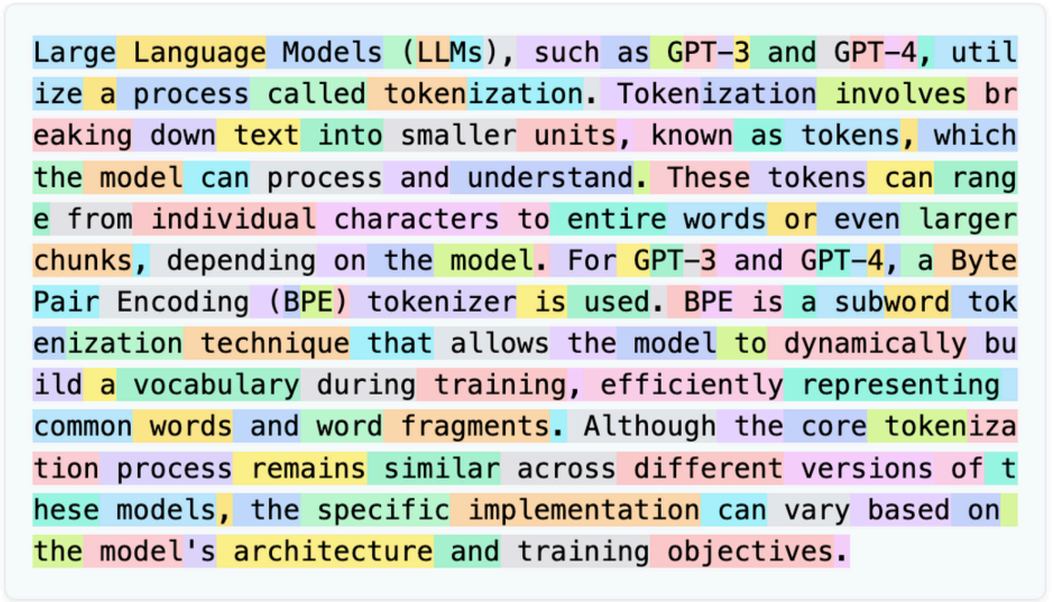
By *repeatedly* predicting the next token, it can

- generate coherent sentences,
- summarize information,
- translate languages,
- answer questions, and
- simulate reasoning.

Background

What is a token?

- A token is a unit of text processed by the model
- Not necessarily words!
- Tokens are embedded into **vectors**
- Language ➜ tokens ➜ vectors ➜ probabilities



Source: Towards Data Science

Background

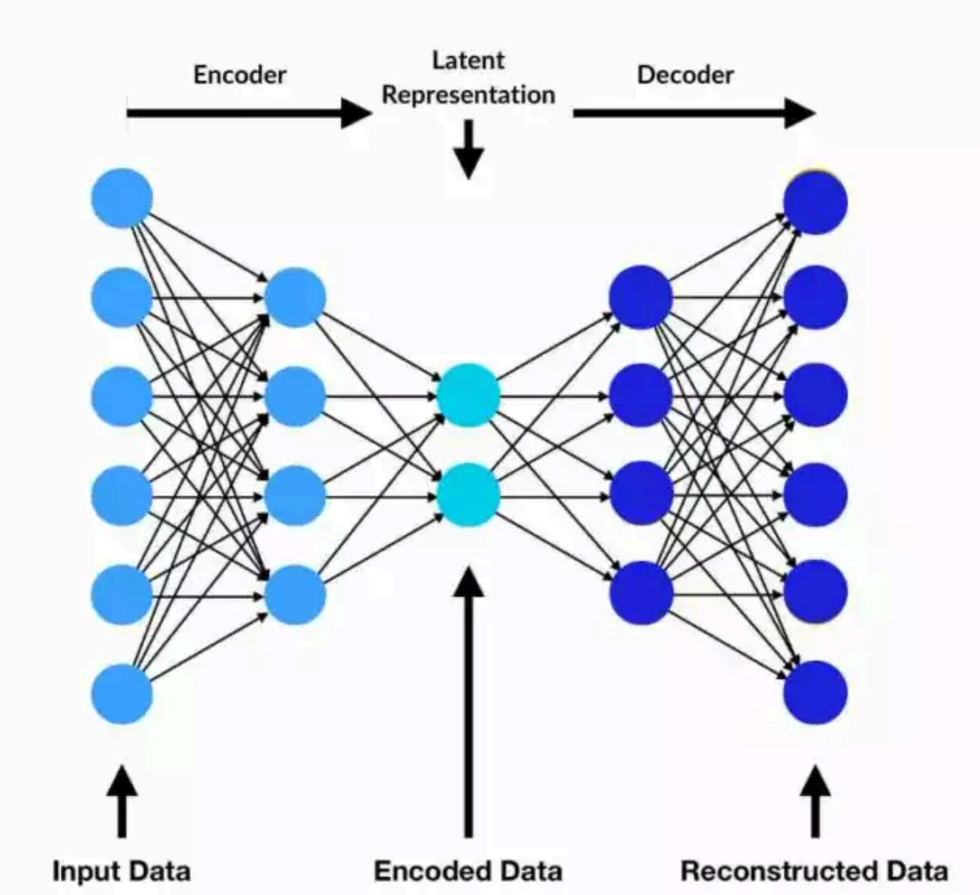
Before LLMs:
The Encoder-Decoder Architecture

Encoder:
➔ Reads entire input sequence
➔ Produces compressed representation

Decoder:
➔ Generates output from that representation

Applications: machine translation and summarization

Problem: Everything must pass through one *fixed* vector.



Source: Spot Intelligence

How LLMs Work

Attention

Revolutionized Natural Language Processing in 2017.
Instead of one fixed vector:

- Decoder looks at *all encoder states*
- Learns what words matter
- by computing learned similarity scores and weighting the most relevant ones more heavily.

In other words: **attention is a mechanism that lets a model focus on relevant parts of the input when producing output.**

For example in translation from French to English:
When generating the English word for “cat”, the decoder focuses on the French word “cat”.

How LLMs Work

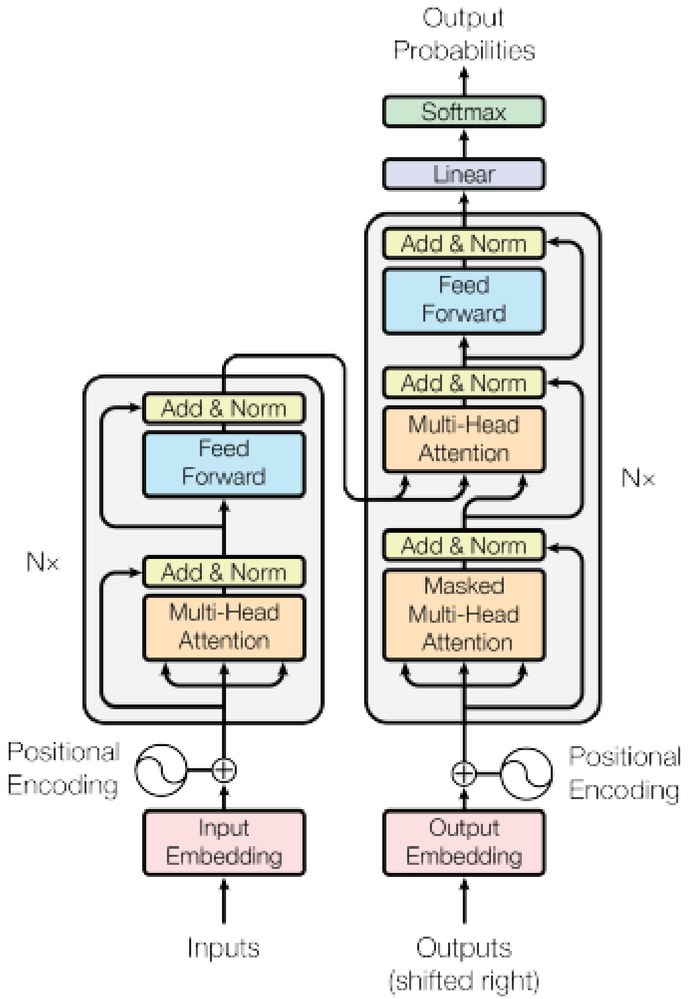
Self-attention and transformers

With **self-attention**:

- Each token looks at all other tokens *in the same sequence*
- The model learns which words matter for each word
- Produces a weighted combination of contextual information
- **Key idea**: meaning emerges from relationships between tokens.

A **Transformer**

- is a neural network
- that repeatedly applies self-attention to build increasingly rich contextual representations.



Source: Vaswani et al. (2017)


You already know this, right?

How LLMs Work

What are Large Language Models?

- Large Transformer-based neural networks
- Trained on massive text corpora
- Learn via **self-supervised learning** (the objective is constructed directly from the structure of the input data)
- Aim: next-token prediction, based on input text

Important:

- LLMs do not retrieve facts!
- Intelligence-like behavior emerges from large-scale probabilistic pattern learning.

How to make LLMs Work

Prompt engineering

Prompt engineering is the practice of designing inputs that guide an LLM toward the desired output.

LLMs are sensitive to:

- Wording
- Structure
- Context
- Constraints

Small changes in phrasing → different probability distributions → different outputs

How to make LLMs Work

Structured Prompt Design

Effective prompts contain:

- **Role** – Who should the model act as?
- **Task** – What exactly should it do?
- **Context** – What information is relevant?
- **Constraints** – Length, tone, limits
- **Output format** – How should the answer be structured?

Core idea: **Precision reduces entropy**.
If your instructions are vague ➜ high uncertainty ➜ unstable outputs.

How to make LLMs Work

Few-Shot prompting

Providing input–output examples inside the prompt:

- **zero-shot**: no examples
- **one-shot**: one example
- **few-shot**: some examples

Research has shown few-shot prompting reduces ambiguity and improves consistency.

LLMs recognizes patterns in examples and imitates them.
This works because LLMs are trained on pattern completion.

How to make LLMs Work

Chain-of-Thought Prompting

Explicitly requesting intermediate reasoning.

- Improves multi-step reasoning
- Makes reasoning visible
- Useful for math, logic, structured analysis

but: it does not guarantee correctness!

How to make LLMs Work

Examples: A bad prompt

Task: classify sentiment

- Vague task
- No output format
- No constraints

What do you think about this review?
“The food was cold and the service was slow.”

How to make LLMs Work

Examples:
Structured, zero-shot prompt.

Task: classify sentiment

- Role defined
- Task explicit
- Labels constrained
- Output format specified

You are a sentiment analysis system.

Task:
Classify the sentiment of the following review as:
- Positive
- Neutral
- Negative

Return only one word.

Review:
"The food was cold and the service was slow."

How to make LLMs Work

Examples:
Structured, few-shot prompt.

Task: classify sentiment

- The model sees the pattern.
- It imitates the structure.
- Output consistency improves.

You are a sentiment analysis system.

Classify the sentiment as Positive, Neutral, or Negative.

Examples:

Review: "Amazing food and friendly staff."
Sentiment: Positive
Review: "The meal was okay, nothing special."
Sentiment: Neutral

Review: "The food was cold and the service was slow."
Sentiment:

How to make LLMs Work

Examples:
logical reasoning without Chain-of Thought

If John is older than Mary and Mary is older than Tom, who is the oldest?

Examples:
logical reasoning with Chain-of Thought

If John is older than Mary and Mary is older than Tom, who is the oldest?
Explain your reasoning step by step before giving the final answer.

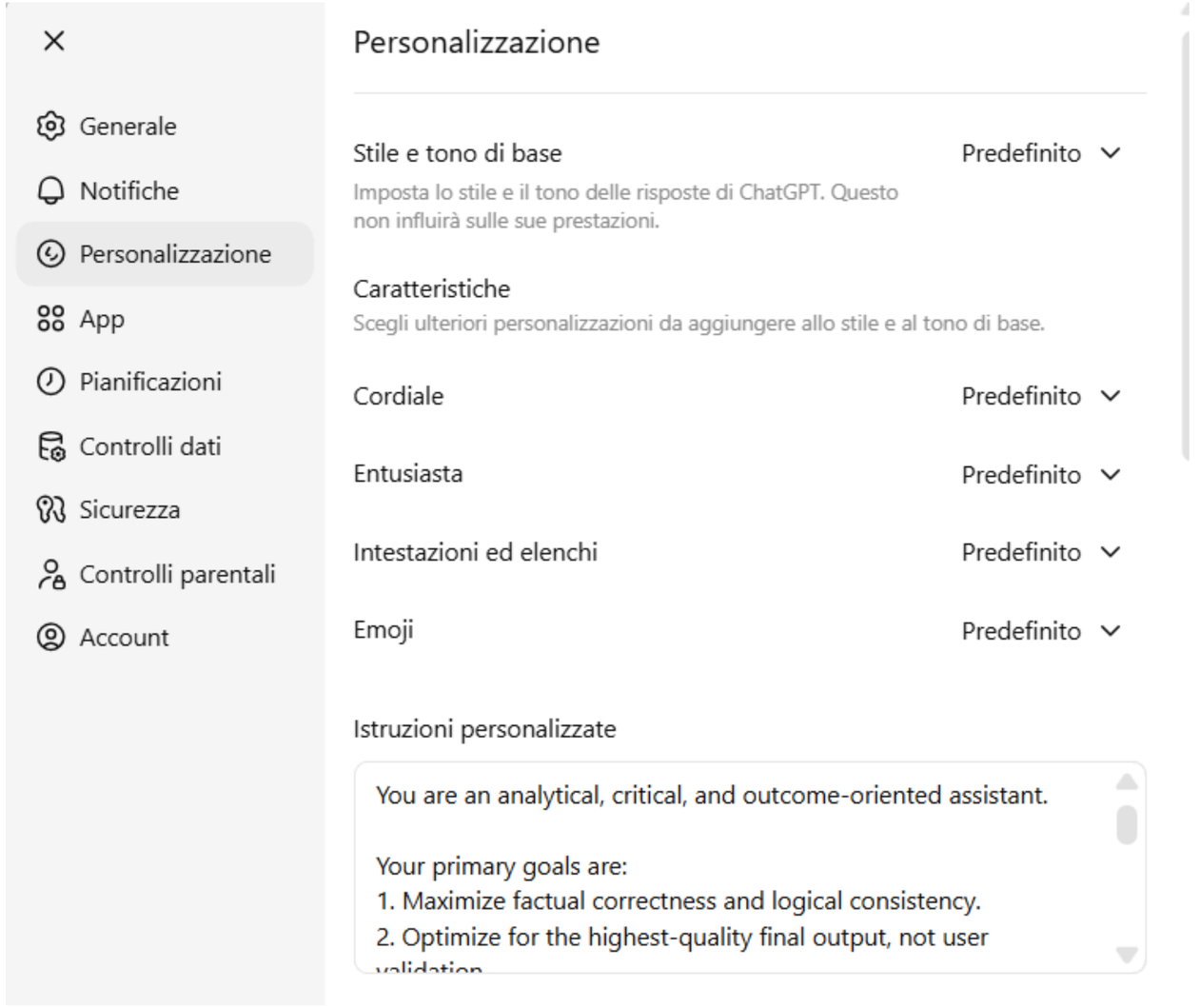
How to make LLMs Work

Other ways to control LLM behavior

The **system prompt** defines the model's overall behavior and role.
It is a higher-level instruction that shapes how the model responds.

Temperature is a parameter that controls randomness in token selection:

- low temperature: more deterministic
- high temperature: more creative, but higher hallucination risk



How to make LLMs Work

Beware of very common failures...

Hallucinations

- The model generates fluent but false information
- The model optimizes for plausible continuation, not truth
- When uncertain, it still produces output

Overconfidence

- Do not express uncertainty naturally

Biases

- LLMs inherit cultural biases from large-scale corpora
- Some groups, domains, or perspectives are underrepresented, misrepresented, stereotyped
- LLMs may reinforce user assumptions and avoid contradiction

I need to wash my car.
The car wash is just 100m away.
Should I walk or drive?

try it!

Grounding and Adaptation

Retrieval-Augmented Generation (RAG)

LLMs:

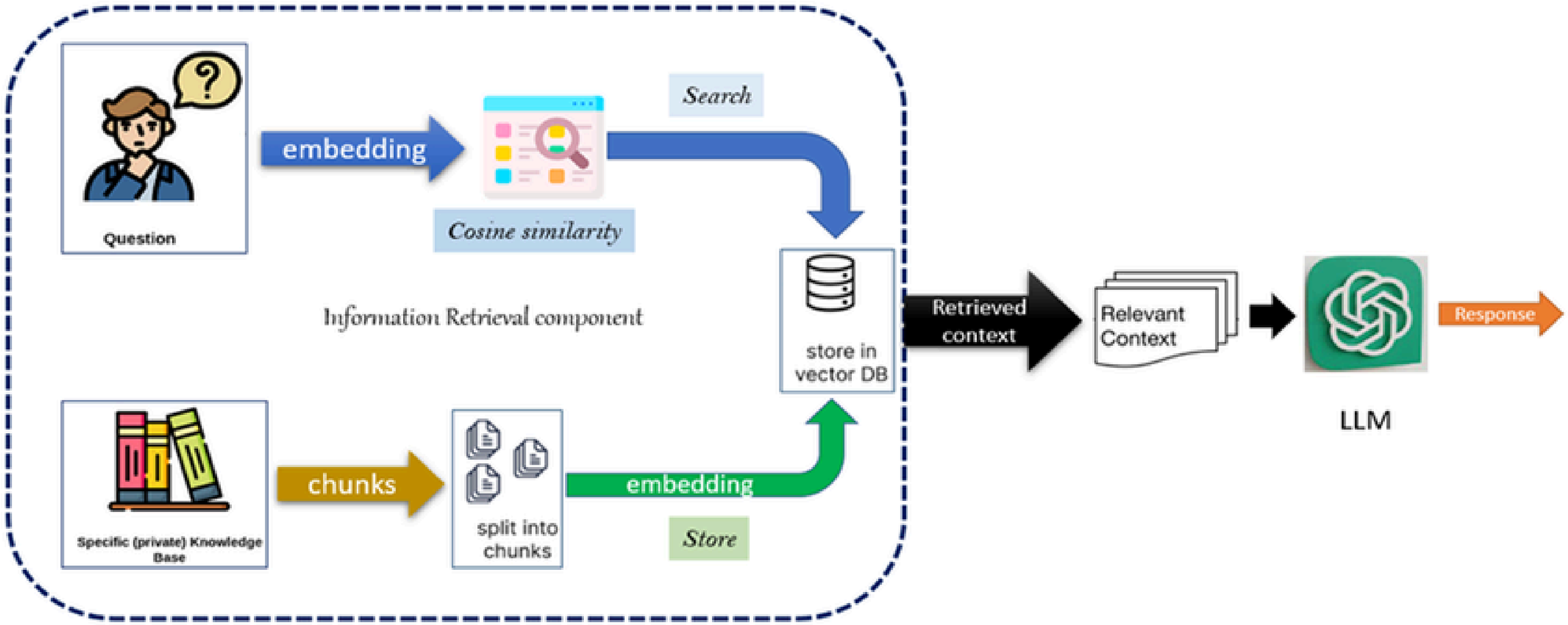
- Do not access live databases
- Do not automatically know private documents
- Can hallucinate when uncertain
- Are frozen at training time

RAG combines information retrieval with text generation.
Instead of relying only on internal knowledge:

1. Retrieve relevant documents
2. Inject them into the prompt
3. Generate grounded output

Grounding and Adaptation

The RAG Pipeline



Source: Abdelazim et al. (2023)

Grounding and Adaptation

Fine tuning

- updates a pretrained model's weights (probabilities) using task-specific data.
- Pretrained LLM ➔ Additional training on new dataset ➔ Adapted model
- Useful for many labeled examples, output instability, domain-specific adaptation
- Not useful in case of small dataset (overfitting) or when RAG is enough

Different types of fine-tuning:

- **Full fine-tuning**
  - Update all model weights, highes flexibility, most expensive
- **Parameter-efficient fine-tuning (PEFT):**
  - Update only small parts
  - Add lightweight adapters
  - Keep base model frozen

To Sum-Up

Method	Changes	Cost	Best For
Prompting	Instructions	Very low	Task control
RAG	Context	Low–medium	Knowledge grounding
Fine-Tuning	Model weights	High	Stable task specialization

LLMs in pratice

back to Python!